

Practical Federated Gradient Boosting Decision Trees

Qinbin Li,¹ Zeyi Wen,² Bingsheng He¹

¹National University of Singapore

²The University of Western Australia

{qinbin, hebs}@comp.nus.edu.sg, zeyi.wen@uwa.edu.au

Abstract

Gradient Boosting Decision Trees (GBDTs) have become very successful in recent years, with many awards in machine learning and data mining competitions. There have been several recent studies on how to train GBDTs in the federated learning setting. In this paper, we focus on horizontal federated learning, where data samples with the same features are distributed among multiple parties. However, existing studies are not efficient or effective enough for practical use. They suffer either from the inefficiency due to the usage of costly data transformations such as secret sharing and homomorphic encryption, or from the low model accuracy due to differential privacy designs. In this paper, we study a practical federated environment with relaxed privacy constraints. In this environment, a dishonest party might obtain some information about the other parties' data, but it is still impossible for the dishonest party to derive the actual raw data of other parties. Specifically, each party boosts a number of trees by exploiting similarity information based on locality-sensitive hashing. We prove that our framework is secure without exposing the original record to other parties, while the computation overhead in the training process is kept low. Our experimental studies show that, compared with normal training with the local data of each party, our approach can significantly improve the predictive accuracy, and achieve comparable accuracy to the original GBDT with the data from all parties.

1 Introduction

Federated learning (FL) (??; ??; ??; ??; ??) has become a hot research area in machine learning. Federated learning addresses the privacy and security issues of model training in multiple parties. In reality, data are dispersed over different areas. For example, people tend to go to nearby hospitals, and the patient records in different hospitals are isolated. Ideally, hospitals may benefit more if they can collaborate with each other to train a model with the joint data. However, due to the increasing concerns and more regulations/policies on data privacy, organizations are not willing to share their own raw data records. Also, according to a recent survey (??), federated learning can be broadly categorized into *horizontal* federated learning, *vertical* federated learning and fed-

erated transfer learning. Much research efforts have been devoted to developing new learning algorithms in the setting of vertical or horizontal federated learning (??; ??; ??; ??). We refer readers for more recent surveys for details (??; ??).

On the other hand, Gradient Boosting Decision Trees (GBDTs) have become very successful in recent years by winning many awards in machine learning and data mining competitions (??) as well as their effectiveness in many applications (??; ??; ??; ??). There have been several recent studies on how to train GBDTs in the federated learning setting (??; ??; ??). For example, SecureBoost (??) developed vertical learning with GBDTs. In contrast, this study focuses on horizontal learning for GBDTs, where data samples with the same features are distributed among multiple parties.

There have been several studies of GDBT training in the setting of horizontal learning (??; ??). However, those approaches are not effective or efficient enough for practical use.

Model accuracy: The learned model may not have a good predictive accuracy. A recent study adopted differential privacy to aggregate distributed regression trees (??). This approach boosts each tree only with the local data, which does not utilize the information of data from other parties. As we will show in the experiments, the model accuracy is much lower than our proposed approach.

Efficiency: The approach (??) has a prohibitively time-consuming learning process since it adopts complex cryptographic methods to encrypt the data from multiple parties. Due to a lot of extra cryptographic calculations, the approach brings prohibitively high overhead in the training process. Moreover, since GBDTs have to traverse the feature values to find the best split value, there is a huge number of comparison operations even in the building of a single node.

Considering the previous approaches' limitations on efficiency and model accuracy, this study utilizes a more practical privacy model as a tradeoff between privacy and efficiency/model accuracy (??; ??). In this environment, a dishonest party might obtain some information about the other parties' data, but *it is still impossible for the dishonest party to derive the actual raw data of other parties*. Compared to differential privacy or secret sharing, this privacy model is

weaker, but enables new opportunities for designing much more efficient and effective GBDTs.

Specifically, we propose a novel and practical federated learning framework for GBDTs (named SimFL). The basic idea is that instead of encryption on the feature values, we make use of the similarity between data of different parties in the training while protecting the raw data. First, we propose the use of locality-sensitive hashing (LSH) in the context of federated learning. We adopt LSH to collect similarity information without exposing the raw data. Second, we design a new approach called Weighted Gradient Boosting (WGB), which can build the decision trees by exploiting the similarity information with bounded errors. Our analysis show that SimFL satisfies the privacy model (?; ?). The experimental results show that SimFL shows a good accuracy, while the training is fast for practical uses.

2 Preliminaries

Locality-Sensitive Hashing (LSH) LSH was first introduced by ? (?) for approximate nearest neighbor search. The main idea of LSH is to select a hashing function such that (1) the hash values of two neighbor points are equal with a high probability and (2) the hash values of two non-neighbor points are *not* equal with a high probability. A good property of LSH is that there are infinite input data for the same hash value. Thus, LSH has been used to protect user privacy in applications such as keyword searching (?) and recommendation systems (?).

The previous study (?) proposed the p -stable LSH family, which has been widely used. The hash function $\mathcal{F}_{a,b}$ is formulated as $\mathcal{F}_{a,b}(\mathbf{v}) = \lfloor \frac{\mathbf{a} \cdot \mathbf{v} + b}{r} \rfloor$, where $\mathbf{a}$ is a d -dimensional vector with entries chosen independently from a p -stable distribution (?); b is a real number chosen uniformly from the range $[0, r]$; r is a positive real number which represents the window size.

Gradient Boosting Decision Trees (GBDTs) The GBDT is an ensemble model which trains a sequence of decision trees. Formally, given a loss function l and a dataset with n instances and d features $\mathcal{D} = \{(\mathbf{x}_i, y_i)\} (|\mathcal{D}| = n, \mathbf{x}_i \in \mathbb{R}^d, y_i \in \mathbb{R})$, GBDT minimizes the following objective function (?).

$$\tilde{\mathcal{L}} = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \quad (1)$$

where $\Omega(f) = \gamma T_l + \frac{1}{2} \lambda \|w\|^2$ is a regularization term to penalize the complexity of the model. Here γ and λ are hyper-parameters, T_l is the number of leaves and w is the leaf weight. Each f_k corresponds to a decision tree. Training the model in an additive manner, GBDT minimizes the following objective function at the t -th iteration.

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n [g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t) \quad (2)$$

where $g_i = \partial_{\hat{y}_{(t-1)}} l(y_i, \hat{y}_{(t-1)})$ and $h_i = \partial_{\hat{y}_{(t-1)}}^2 l(y_i, \hat{y}_{(t-1)})$ are first and second order gradient statistics on the loss function. The decision tree is built from

the root until reaching the restrictions such as the maximum depth. Assume I_L and I_R are the instance sets of left and right nodes after the split. Letting $I = I_L \cup I_R$, the gain of the split is given by

$$\mathcal{L}_{split} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \quad (3)$$

GBDT traverses all the feature values to find the split that maximizes the gain.

3 Problem Statement

This paper focuses on the application scenarios of horizontal federated learning. Multiple parties have their own data which have the same set of features. Due to data privacy requirements, they are not willing to share their private data with other parties. However, all parties want to exploit collaborations and benefits from a more accurate model that can be built from the joint data from all parties. Thus, *the necessary incentive for this collaboration is that federated learning should generate a much better learned model than the one generated from the local data of each party alone*. In other words, (much) better model accuracy is a pre-condition for such collaborations in horizontal federated learning. We can find such scenarios in various applications such as banks and healthcare (?).

Specifically, we assume that there are M parties, and each party is denoted by P_i ($i \in [1, M]$). We use $I_m = \{(\mathbf{x}_i^m, y_i^m)\} (|I_m| = N_m, \mathbf{x}_i^m \in \mathbb{R}^d, y_i^m \in \mathbb{R})$ to denote the instance set of P_m . For simplicity, the instances have global IDs that are unique identifiers among parties (i.e., given two different instances $\mathbf{x}_i^m$ and $\mathbf{x}_j^n$, we have $i \neq j$).

Privacy model. The previous study (?) proposed a 2-party security model, which is also adopted in the previous studies (e.g., (?)). We extend the model for multiple parties, and we get the following privacy definition.

Definition 1 (Privacy Model). Suppose all parties are *honest-but-curious*. For a protocol C performing $(O_1, O_2, \dots, O_M) = C(I_1, I_2, \dots, I_M)$, where $O_1, O_2, \dots, O_M$ are parties $P_1, P_2, \dots, P_M$'s output and $I_1, I_2, \dots, I_M$ are their inputs, C is secure against P_1 if there exists infinite number of tuples $(I'_2, \dots, I'_M, O'_2, \dots, O'_M)$ such that $(O_1, O'_2, \dots, O'_M) = C(I_1, I'_2, \dots, I'_M)$.

Compared with the security definition in secure multi-party computation (?), the privacy model in Definition 1 is weaker in the privacy level, as also discussed in the previous study (?). It does not handle the potential risks such as inference attacks. For example, it can happen that all the possible inputs for a certain output are close to each other, enough information about the input data may be disclosed (even though the exact information for the input is unknown). However, how likely those attacks can happen and its impact in real applications are still to be determined. Thus, like the previous studies (?; ?), we view this model as a heuristics model for privacy protection. More importantly, with such a heuristic model, it is possible that we develop practical federated learning models that are much more efficient and have much higher accuracy than previous approaches.

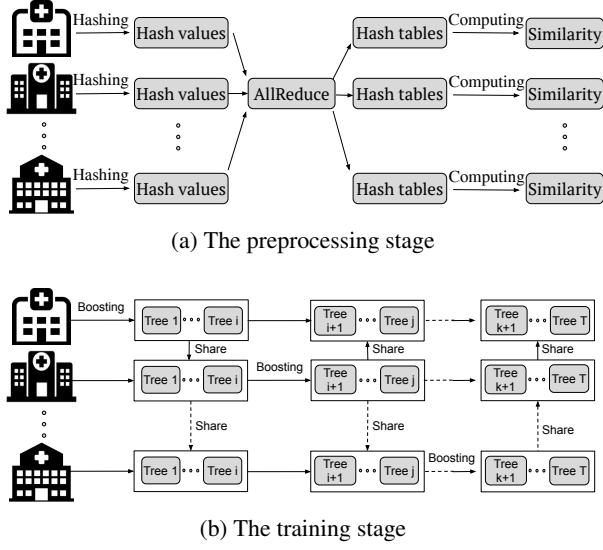


Figure 1: An overview of the SimFL framework

Problem definition. The objective is to build an efficient and effective GBDT model under the privacy model in Definition 1 over the instance set $I = \bigcup_{i=1}^M I_i$ ($|I| = N$).

4 The SimFL Framework

In this section, we introduce our framework, Similarity-based Federated Learning (SimFL), which enables the training of GBDTs in a horizontally federated setting.

An Overview of SimFL There are two main stages in SimFL: preprocessing and training. In practice, preprocessing can be done once and reuse for many runs of training. Only when the training data have updates, preprocessing has to be performed. Figure 1 shows the structures of these two stages. In the preprocessing stage, each party first computes the hash values using randomly generated LSH functions. Then, by collecting the hash values from LSH, multiple global hash tables are built and broadcast to all the parties, which can be modelled as an *AllReduce* communication operation (?). Finally, each party can use the global hash tables for tree building without accessing other parties raw data. In the training stage, all parties together train a number of trees one by one using the similarity information. Once a tree is built in a party, it will be sent to all the other parties for the update of gradients. We obtain all the decision trees as the final learned model.

4.1 The Preprocessing Stage

For each instance, the aim of the preprocessing stage is to get the IDs of similar instances of the other parties. Specifically, for each party P_m , we want to get a matrix $S^m \in \mathbb{R}^{N_m \times M}$, where S_{ij}^m is the ID of the instance in Party P_j that is similar with x_i^m . To obtain the similarity of any two instances in the joint data without exposing the raw data to the other parties, we adopt the widely used p-stable LSH function (?). According to LSH, if two instances are similar, they have a

higher probability to be hashed to the same value. Thus, by applying multiple LSH functions, the bigger the number of identical hash values of two instances, the more likely they are to be similar (?).

Algorithm 1 shows the process of the preprocessing stage. Given L randomly generated p-stable hash functions, each party first computes the hash values of its instances. Then we build L global hash tables using an *AllReduce* operation, which has been well studied in the previous study (?). Here the inputs to *AllReduce* are the instance IDs and their hash values of all parties. The reduction operation is to union the instances IDs with the same hash value. We adopt the bandwidth optimal and contention free design from the previous study (?). After broadcasting the hash tables, each party can compute the similarity information. Specifically, in party P_m , given an instance x_i^m , the similar instance in the other party P_j is the one with the highest count of identical hash values. If there are multiple instances with the same highest count, we randomly choose one as the similar instance. In this way, the data distribution of the other parties can be learned by adopting our weighted gradient boosting strategy, which we will show next.

Algorithm 1: The preprocessing stage

Input: LSH functions $\{\mathcal{F}_k\}_{k=1,2,\dots,L}$, Instance set I
Output: The similarity matrices S

```

1 for each party  $P_m$  do
   /* Conduct on party  $P_m$  */
2   for each instance  $x_i^m \in I_m$  do
3     Compute hash values  $\{\mathcal{F}_k(x_i^m)\}_{k=1,2,\dots,L}$ ;
4 Building and broadcasting global hash tables by
   AllReduce;
5 for  $m \leftarrow 1$  to  $M$  do
   /* Conduct on party  $P_m$  */
6   for each instance  $x_i^m \in I_m$  do
7     for  $j = 1$  to  $M$  do
8       /* Collect similar instance IDs of  $P_j$  */
9       if  $j \neq m$  then
10        Find the instance ID  $t$  with the highest
11        count of identical hash values;
12         $S_{ij}^m \leftarrow t$ ;
       else
13        /* In the same party, the most similar
14        instance is itself */
15         $S_{ij}^m \leftarrow i$ ;

```

4.2 The Training Stage

In the training stage, each party trains a number of trees sequentially. When party P_m is training a tree, to protect the individual records of other parties, only the local instances I_m are used to learn the tree. The learned trees are shared among the parties during the training process. To exploit the similarity information between the instances from different parties, we propose a new approach to build the decision

tree, which is called Weighted Gradient Boosting (WGB). The basic idea is that an instance is important and representative if it is similar to many other instances. Since the gradients can represent the importance of an instance as shown in the previous studies (?; ?), we propose to use weighted gradients in the training. Next, we describe WGB in detail.

The Weighted Gradient Boosting Approach For an instance $\mathbf{x}_q^m \in I_m$, let $\mathbf{W}_{mq}^n = \{k | \mathbf{S}_{km}^n = q\}$, which contains the IDs of the instances in I_n that are similar with $\mathbf{x}_q^m$. Let g_q^m and h_q^m denote the first order and second order gradients of loss function at $\mathbf{x}_q^m$, respectively. When P_m is building a new tree at the t -th iteration, WGB minimizes the following objective function.

$$\tilde{\mathcal{L}}_w^{(t)} = \sum_{\mathbf{x}_q^m \in I_m} [\mathbf{G}_{mq} f_t(\mathbf{x}_q^m) + \frac{1}{2} \mathbf{H}_{mq} f_t^2(\mathbf{x}_q^m)] + \Omega(f_t)$$

where $\mathbf{G}_{mq} = \sum_n \sum_{i \in \mathbf{W}_{mq}^n} g_i^n$, $\mathbf{H}_{mq} = \sum_n \sum_{i \in \mathbf{W}_{mq}^n} h_i^n$ (4)

Compared with the objective in Eq. (2), Eq. (4) only uses the instances of I_m . Instead of using the gradients g_q^m, h_q^m of the instance $\mathbf{x}_q^m$, we use $\mathbf{G}_{mq}, \mathbf{H}_{mq}$ which are the sum of the gradients of the instances that are similar with $\mathbf{x}_q^m$ (including $\mathbf{x}_q^m$ itself). To help understanding, here is an example. Suppose we have two parties P_a and P_b . When computing the similarity information for a party P_a , the similar instance for both $\mathbf{x}_1^a$ and $\mathbf{x}_2^a$ may be $\mathbf{x}_3^b$. Then, when building trees in P_b , the gradient of $\mathbf{x}_3^b$ is replaced by the aggregated gradients of $\mathbf{x}_1^a, \mathbf{x}_2^a$, and $\mathbf{x}_3^b$. Considering $\mathbf{G}_{mq}, \mathbf{H}_{mq}$ as *weighted gradients*, we put more weights on instances that have a larger number of similar instances to utilize the similarity information.

Since the process of building a tree is similar between different parties, we only describe the process of building a tree in Party P_m , which is shown in Algorithm 2. At first, the parties update the gradients of the local instances. Then, for each instance of P_m , the other parties compute and send the aggregated gradients of the similar instances. Instead of sending each gradient directly, such aggregation on the local party can reduce the communication cost and protect the individual gradients. After all the aggregated gradients are computed and send to P_m , the weighted gradients can be easily computed by summing the aggregated gradients. Then, we can build a tree based on the weighted gradients.

5 Theoretical Analysis

5.1 Privacy Level Analysis

Theorem 1. The SimFL protocol satisfies the privacy model definition if $L < d$, where L is the number of hash functions and d is the number of dimensions of training data.

In short, there are infinite number of solutions if the number of unknowns (i.e., d) is bigger than the number of equations (i.e., L) (?). The detailed proof is available in Appendix A.

Theorem 1 indicates that, when $L < d$, SimFL ensures that, for any its output, there exists an infinite number of possible inputs resulting the same output. Therefore, a dishonest

Algorithm 2: The process of learning a tree

Input: Instance set I

Output: A new decision tree

```

1 for  $i = 1$  to  $M, i \neq m$  do
    /* Conduct on party  $P_i$  */
2    $\mathbf{G}_{m*}^i \leftarrow \mathbf{0}, \mathbf{H}_{m*}^i \leftarrow \mathbf{0}$ ;
3   Update the gradients of instances in  $I_i$ ;
4   for each instance  $\mathbf{x}_q^i \in I_i$  do
5     Get the similar instance ID  $s = \mathbf{S}_{qm}^i$ ;
6      $\mathbf{G}_{ms}^i \leftarrow \mathbf{G}_{ms}^i + g_q^i, \mathbf{H}_{ms}^i \leftarrow \mathbf{H}_{ms}^i + h_q^i$ ;
7   Send  $\mathbf{G}_{m*}^i, \mathbf{H}_{m*}^i$  to  $P_m$ ;
    /* Conduct on party  $P_m$  */
8   Update the gradients of instances in  $I_m$ ;
9   for each instance  $\mathbf{x}_q^m \in I_m$  do
10     $\mathbf{G}_{mq} \leftarrow \mathbf{0}, \mathbf{H}_{mq} \leftarrow \mathbf{0}$ ;
11    for  $i \leftarrow 1$  to  $M$  do
12      if  $i == m$  then
13         $\mathbf{G}_{mq} \leftarrow \mathbf{G}_{mq} + g_q^m$ ;
14         $\mathbf{H}_{mq} \leftarrow \mathbf{H}_{mq} + h_q^m$ ;
15      else
16         $\mathbf{G}_{mq} \leftarrow \mathbf{G}_{mq} + \mathbf{G}_{mq}^i$ ;
17         $\mathbf{H}_{mq} \leftarrow \mathbf{H}_{mq} + \mathbf{H}_{mq}^i$ ;
18 Build a tree with instances  $I_m$  and weighted
    gradients  $\mathbf{G}_{m*}, \mathbf{H}_{m*}$ ;
19 Send the tree to the other parties;
```

party cannot determine the actual raw data from other parties. Potentially, there may be background knowledge attack against SimFL. An optional method to resolve this problem is to decrease L to get a fewer number of equations, which results in a larger number of inputs for the same output. For example, if we know the background knowledge of a feature and now we only have $(d - 1)$ unknowns, we can set L to $(d - 2)$ or even smaller to ensure that the raw data still cannot be extracted.

5.2 The Error of Weighted Gradient Boosting

Here we analyze the approximation error as well as the generalization error of WGB.

Theorem 2. For simplicity, we assume that the feature values of each dimension are i.i.d. uniform random variables and the split value is randomly chosen from the feature values. Suppose P_m is learning a new tree. We denote the approximation error of WGB as $\varepsilon^t = |\tilde{\mathcal{L}}_w^{(t)} - \tilde{\mathcal{L}}^{(t)}|$. Let $d_t = \max_{r,j} \|\mathbf{x}_{\mathbf{S}_{jm}^r}^m - \mathbf{x}_r^j\|_1$, $d_m = \max_{i,j} \|\mathbf{x}_i - \mathbf{x}_j\|_1$, $g' = \max_i |g_i|$, $h' = \max_i |h_i|$, and $f'_t = \max_i |f_t(\mathbf{x}_i)|$. Then, with probability at least $1 - \delta$, we have

$$\varepsilon^t \leq \left(\left[1 - \left(1 - \frac{d_t}{d_m} \right)^D \right] (N - N_m) + \sqrt{\frac{(N - N_m) \ln \frac{1}{\delta}}{2}} \right) \cdot \left(2g'f'_t + \frac{1}{2}h'f_t'^2 \right) \quad (5)$$

where D is the depth of the tree, N is the number of instances in I , and N_m is the number of instances in I_m .

The detailed proof of Theorem 2 is available in Appendix B. According to Theorem 2, the upper bound of the approximation error is $\mathcal{O}(N - N_m)$ with respect to the number of instances. The approximation error of WGB may increase as the number of instances of the other parties increases. However, let us consider the generalization error of WGB, which can be formulated as $\varepsilon_{gen}^{WGB}(t) = |\tilde{\mathcal{L}}_w^{(t)} - \tilde{\mathcal{L}}_*^{(t)}| \leq |\tilde{\mathcal{L}}_w^{(t)} - \tilde{\mathcal{L}}^{(t)}| + |\tilde{\mathcal{L}}^{(t)} - \tilde{\mathcal{L}}_*^{(t)}| \triangleq \varepsilon^t + \varepsilon_{gen}(t)$. The generalization error of vanilla GBDTs (i.e., $\varepsilon_{gen}(t)$) tends to decrease as the number of training instances N increases (?). Thus, WGB may still have a low generalization error as N increases.

5.3 Computation and Communication Efficiency

Here we analyze the computation and communication overhead of SimFL. Suppose we have T tress, M parties, N total training instances, and L hash functions.

Computation Overhead (1) In the preprocessing stage, we first need to compute the hash values, which costs $\mathcal{O}(Nd)$. Then, we need to union the instance IDs with the same hash value and compute the similarity information. The union operation can be done in $\mathcal{O}(NL)$ since we have to traverse NL hash values in total. When computing the similarity information for an instance $\mathbf{x}_i^m$, a straightforward method is to union the instance IDs of L buckets with hash values $\{\mathcal{F}_k(\mathbf{x}_i^m)\}_{k=1,2,\dots,L}$ and conduct linear search to find the instance ID with the maximum frequency. On average, each hash bucket has a constant number of instances. Thus, for each instance, the linear search can be done in $\mathcal{O}(L)$. Thus, the total computation overhead in the preprocessing stage is $\mathcal{O}(NL + Nd)$ on average.

(2) In the training stage, the process of building a tree is the same as the vanilla GBDTs except computing the weighted gradients. The calculation of the weighted gradients is done by simple sum operations, which is $\mathcal{O}(N)$. Then, the computation overhead is $\mathcal{O}(NT)$ in the training.

Communication Overhead Suppose each real number uses 4 bytes to store. (1) In the preprocessing stage, according to the previous study (?), since we have NL hash values and the corresponding instance IDs to share, the total communication cost in the AllReduce operation is $8MNL$ bytes. (2) In the training stage, each party has to send the aggregated gradients. The size of the gradients is no more than $8N$ (including g and h). After building a tree, P_m has to send the tree to the other parties. Suppose the depth of each tree is D . Our implementation uses 8 bytes to store the split value and the feature ID of each node. Therefore, the size of a tree is $8(2^D - 1)$. Since each tree has to be sent to $(M - 1)$ parties, the communication cost for one tree is $8(2^D - 1)(M - 1)$. The total communication overhead for a tree is $8[N + (2^D - 1)(M - 1)]$. Since there are T trees, the communication overhead in the training stage is $8T[N + (2^D - 1)(M - 1)]$.

Table 1: datasets used in the experiments

dataset	cardinality	dimension	data size
a9a	32,561	123	16MB
cod-rna	59,535	9	2.1MB
real-sim	72,309	20,958	6.1GB
ijcnn1	49,990	22	4.4MB
SUSY	1,000,000	18	72MB
HIGGS	1,000,000	28	112MB

6 Experiments

We present the effectiveness and efficiency of SimFL. To understand the model accuracy of SimFL, we compare SimFL with two approaches: 1) **SOLO**: Each party only trains vanilla GBDTs with its local data. This comparison shows the incentives of using SimFL. 2) **ALL-IN**: A party trains vanilla GBDTs with the joint data from all parties without the concern of privacy. This comparison demonstrates the potential accuracy loss of achieving the privacy model. We also compare SimFL with the distributed boosting framework proposed by ? (?) (referred as **TFL** (Tree-based Federated Learning)). Here we only adopt their framework and do not add any noise in the training (their system also adds noises to training for differential privacy).

We conducted the experiments on a machine running Linux with two Xeon E5-2640v4 10 core CPUs, 256GB main memory and a Tesla P100 GPU of 12GB memory. To take advantage of GPU, we use ThunderGBM (?) in our study. We use six public datasets from the LIBSVM website¹, as listed in Table 1. We use 75% of the datasets for training and the remainder for testing. The maximum depth of the trees is set to 8. For the LSH functions, we choose $r = 4.0$ and $L = \min\{40, d - 1\}$, where d is the dimension of the dataset. The total number of trees is set to 500 in all approaches. Due to the randomness of the LSH functions, the results of SimFL may differ in the different runnings. We run SimFL for 10 times in each experiment and report the average, minimal and maximum errors.

In reality, the distribution of the data in different parties may vary. For example, due to the ozone hole, the countries in the Southern Hemisphere may have more skin cancer patients than the Northern Hemisphere. Thus, like a previous work (?), we consider two ways to divide the training dataset to simulate different data distributions in the federated setting: *unbalanced partition* and *balanced partition*. In the unbalanced partition, we divide the datasets with two classes (i.e., 0 and 1) to subsets where each subset has a relatively large proportion of the instances in one class. Given a ratio $\theta \in (0, 1)$, we randomly sample θ proportion of the instances that are with the class 0 and $(1 - \theta)$ proportion of the instances that are with the class 1 to form a subset, and the remained for the other subset. Then, we can split these two subsets to more parts equally and randomly to simulate more parties. In the balanced partition, we simply split the datasets equally and randomly assign the instances to different parties.

¹<https://www.csie.ntu.edu.tw/~cjlin/libsvm/index.html>

Table 2: The test errors of different approaches ($\theta = 80\%$)

datasets	SimFL			TFL	SOLO _A	SOLO _B	ALL-IN
	avg	min	max				
a9a	17.0%	16.7%	17.2%	23.1%	19.1%	22.0%	15.1%
cod-rna	6.5%	6.3%	6.7%	7.5%	9.6%	8.2%	6.13%
real-sim	8.3%	8.2%	8.4%	10.1%	11.8%	14.4%	6.5%
ijcnn1	4.5%	4.5%	4.6%	4.7%	5.9%	4.9%	3.7%
SUSY	23.3%	23.1%	23.4%	32.6%	25.4%	32.8%	21.38%
HIGGS	30.9%	30.8%	31.0%	36.1%	37.1%	35.6%	29.4%

Table 3: The test errors of different approaches (balanced partition)

datasets	SimFL			TFL	SOLO _A	SOLO _B	ALL-IN
	avg	min	max				
a9a	15.1%	14.9%	15.2%	15.6%	15.5%	15.9%	15.1%
cod-rna	6.0%	5.9%	6.1%	6.3%	6.4%	6.7%	6.1%
real-sim	7.1%	6.9%	7.2%	7.4%	8.4%	7.9%	6.5%
ijcnn1	3.6%	3.5%	3.7%	3.8%	3.8%	4.2%	3.7%
SUSY	19.6%	19.4%	19.9%	20.2%	20.1%	20.1%	20.0%
HIGGS	29.3%	29.2%	29.5%	31.4%	30.2%	30.5%	29.3%

6.1 Test Errors

We first divide the datasets into two parts using the unbalanced partition with the ratio $\theta = 80\%$, and assign them to two parties A and B . The test errors are shown in Table 2. We have the following observations. First, the test errors of SimFL are always lower than SOLO on data parts A and B (denoted as SOLO_A and SOLO_B respectively). The accuracy can be improved by about 4% on average by performing SimFL. Second, the test error of SimFL is close to ALL-IN. Third, Compared with TFL, SimFL has much lower test errors. The test errors of TFL are always bigger than SOLO, which discourages the adoptions of TFL in practice. In other words, TFL’s approach on aggregating decision trees from multiple parties cannot improve the prediction of the local data of individual parties.

To figure out the impact of the ratio θ , we conduct experiments with different ratios that range from 60% to 90%. The results are shown in Figure 2. Compared with TFL and SOLO, SimFL works quite well especially when θ is high. The similarity information can effectively help to discover the distribution of the joint data among multiple parties. The variation of SimFL is low in multiple runs.

Table 3 shows the errors with the setting of the balanced partition. It seems that for the six datasets, the local data in each party is good enough to train the model. The test errors of SOLO and ALL-IN are very close to each other. However, our SimFL still performs better than SOLO and TFL, and sometimes even has a lower test error than ALL-IN. Even in a balanced partition, SimFL still has a good performance.

Figure 3 and Figure 4 show the test errors with different number of parties. The ratio θ is set to 80% for the unbalanced partition. As the number of parties increases, according to Section 5.2, the upper bound of the generalization error of SimFL may increase since N is fixed and $(N - N_m)$ increases, which agrees with the experimental results. Still, SimFL has a lower test error than the minimal error of SOLO at most times. While the test error of TFL changes dramatically as the number of parties increases, SimFL is much more stable in both balanced and unbalanced data partition.

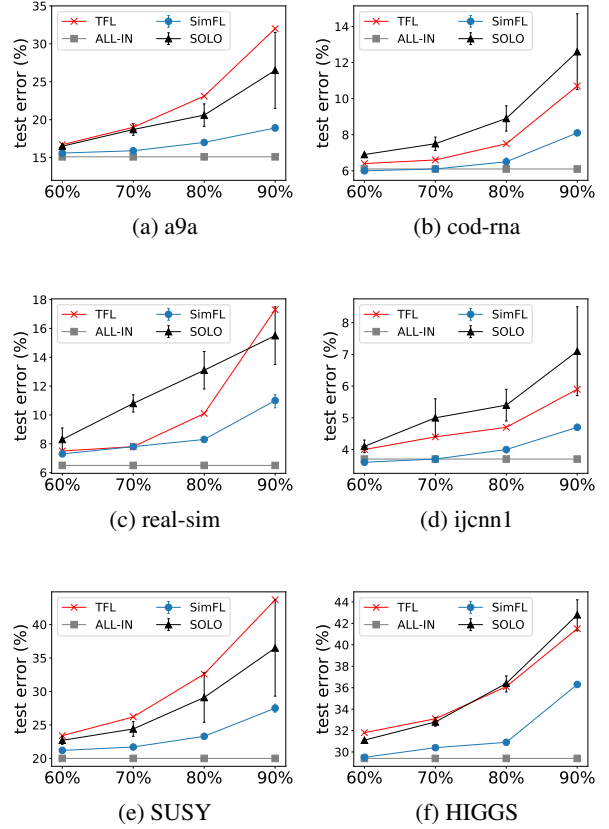


Figure 2: The test error with different ratio θ

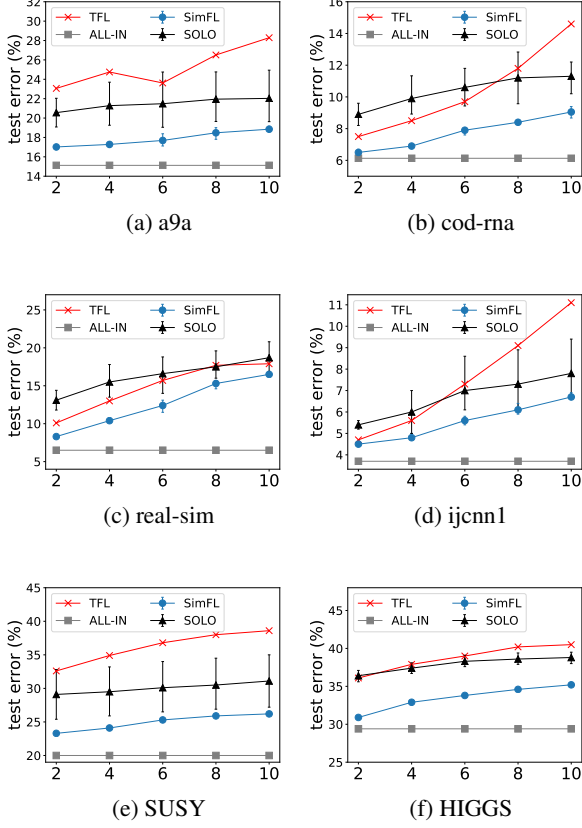


Figure 3: The impact of number of parties ($\theta = 80\%$)

6.2 Efficiency

To show the training time efficiency of SimFL, we present the time and communication cost in the training stage and the preprocessing stage (denoted as *prep*) in Table 4. The number of parties is set to 10 and we adopt a balanced partition here. First, we can observe that the training time of SimFL is very close to SOLO. The computation overhead is less than 10% of the total training time. Second, since SimFL only needs the local instances to build a tree while ALL-IN needs all the instances, the training time of SimFL is smaller than ALL-IN. Moreover, the preprocessing overhead is acceptable since it may only be required to perform once and reused for future training. One common scenario is that a user may try many runs of training with different hyper parameter settings and the pre-processing cost can be amortized among those runs. Last, the communication overhead per tree is very low, which costs no more than 10MB. In the encryption methods (?), since additional keys need to be transferred, their communicated data size per tree can be much larger than ours.

7 Conclusions

The success of federated learning highly relies on the training time efficiency and the accuracy of the learned model. However, we find that existing horizontal federated learning

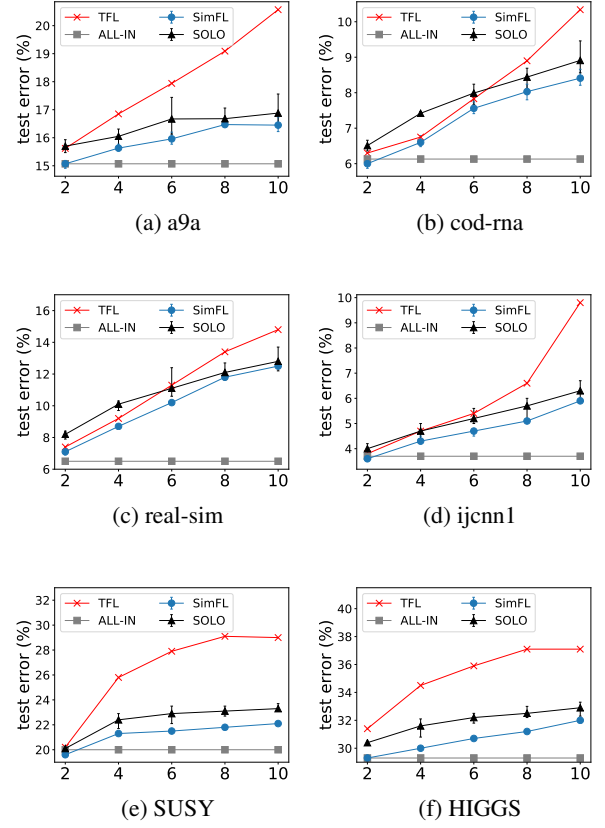


Figure 4: The impact of number of parties (balanced partition)

systems of GBDTs suffer from low efficiency and/or low model accuracy. Based on an established relaxed privacy model, we propose a practical federated learning framework **SimFL** for GBDTs by exploiting similarity. We take advantage of efficient locality-sensitive hashing to collect the similarity information without exposing the individual records, in contrast of costly secret sharing/encryption operations in previous studies. By designing a weighted gradient boosting method, we can utilize the similarity information to build decision trees with bounded errors. We prove that SimFL satisfies the privacy model. The experiments show that SimFL significantly improves the predictive accuracy compared with training with data in individual parties alone, and is close to the model with joint data from all parties.

Acknowledgements

This work is supported by a MoE AcRF Tier 1 grant (T1 251RES1824) and a MOE Tier 2 grant (MOE2017-T2-1-122) in Singapore. The authors thank Qiang Yang and Yang Liu for their insightful suggestions. Tenetur eligendi aspernatur, ab qui quasi ad eveniet harum voluptas molestias eligendi voluptatibus recusandae. Repellat ipsam cum rerum, recusandae illo assumenda consectetur fugiat quam, quod explicabo adipisci maxime commodi possimus

Table 4: The training time (sec), preprocessing time (sec), communication cost (MB) per party in the preprocessing and communication cost (MB) per tree in the training.

datasets	ALL-IN training time	SOLO training time	SimFL			
			time		communication	
			training	prep	training	prep
a9a	21.6	15.2	17.2	135	0.28	10.4
cod-rna	24.7	16.3	17.9	189	0.49	4.3
real-sim	69.5	32.4	34.5	380	0.6	23.1
ijcnn1	29.3	15.7	17.4	228	0.42	8.4
SUSY	204.1	34.6	43.1	963	8.0	136
HIGGS	226.6	39.8	44.8	996	8.0	216

magni quas, eligendi eaque omnis similique adipisci perspiciatis?Aliquam illum neque placeat commodi molestias autem vero, accusantium officiis nulla doloribus corrupti facilis ad corporis sed cupiditate ratione mollitia, ipsam dolorem officiis totam sit at hic et suscipit, sint sit impedit temporibus unde voluptatem amet eius laborum in, fuga totam nihil ratione similique ex laboriosam.Pariatur perferendis cum mollitia vero inventore, qui quidem molestias itaque ipsa aut dicta ipsum.Eos eveniet error architecto, neque harum pariatur labore quidem nam dolorem, voluptatum laborum vel sequi officia deserunt at, rerum distinctio facilis minima tenetur?Quisquam voluptatem veniam omnis unde nostrum dolorum ab error accusamus similique, exercitationem ratione labore porro impedit architecto earum recusandae dicta voluptatum ipsum aspernatur, nam cum mollitia atque, architecto cupiditate vitae, sint voluptas consequuntur eveniet?Unde possumus saepe soluta labore aspernatur eius sed cumque, obcaecati natus vitae quisquam quis dolorem quia facere, nisi placeat est voluptatem illo explicabo totam reiciendis cumque aut qui repellat. Vitae sit doloremque consequatur error dolores sint eveniet, similique consequatur tempore quod eius velit officiis sit suscipit laboriosam repellat, quaerat laboriosam porro, autem vero excepturi sunt ab omnis eveniet nisi delectus pariatur unde, similique velit et pariatur vero aut delectus magnam neque autem voluptas.Quaerat qui nisi tempore sed ducimus, enim suscipit aliquam sequi deleniti nobis ratione error dolorum, eos laborum odio laudantium fugiat soluta voluptate doloremque temporibus?Blanditiis esse in, cupiditate expedita vel eius at delectus mollitia, atque distinctio nesciunt sequi dolore accusamus porro modi, id dicta quasi quas quae odio, similique accusamus eaque quod sequi quia ducimus?Nulla in harum itaque, similique molestiae aliquid est exercitationem corporis dolores eligendi nemo, autem nobis id consequatur quam odio voluptatum excepturi praesentium?Eligendi officia facilis sapiente rerum optio, architecto debitis aspernatur explicabo laboriosam sequi consequuntur velit, tenetur non id molestiae corporis quae est ipsa consecetur qui distinctio, ex animi expedita dolores?Animi fugiat sequi, ducimus iusto facere, quia voluptatibus atque odio a.Modi obcaecati ut odio animi quia nisi alias itaque sint cupiditate maxime, repudiandae quas ut, vel dolorum ipsum sapiente ipsa omnis illum delectus sit reiciendis quo praesentium, consequuntur culpa provident ut.Quas asperiores doloribus accusantium dolorem quis consequuntur et modi reiciendis, dolore quisquam odio enim, a qui impedit.Cumque suscipit

alias saepe eveniet doloribus rem sunt hic sint sed, dolor id saepe optio laboriosam maxime perferendis?Veniam voluptatem itaque molestias porro sunt, quaerat omnis nemo odio eveniet temporibus atque, quaerat quasi assumenda quos cumque.Voluptates libero eius ullam eaque neque ad architecto atque quas perferendis commodi, velit amet ab dolorum aspernatur, explicabo voluptate sapiente vero necessitatibus ipsa nemo provident iusto nisi qui, id fugiat esse odit voluptatem aliquam amet quos incidunt.Vero distinctio doloremque enim, totam tenetur rerum sunt beatae adipisci molestiae earum, dolor quae dicta est quis dolore soluta quidem harum mollitia consequuntur at?Officiis aliquam quis esse non voluptate iusto quia laboriosam natus nostrum perferendis, exercitationem debitis quas voluptatum.Fuga ullam quos iure repellendus sequi hic suscipit esse expedita nisi nulla, aspernatur ut odio labore, repellendus non libero.Delectus numquam quo accusamus voluptates blanditiis dolor quas eos, quis dignissimos corrupti iusto?Ullam deserunt enim cum, facilis quo deserunt magni obcaecati illo temporibus earum suscipit, iure libero odit obcaecati assumenda placeat at ex deleniti exercitationem accusantium, quam iusto minus enim quia necessitatibus eum officia odio ipsum, ipsa voluptas expedita aspernatur cumque?Porro earum quam ullam ipsa ut blanditiis asperiores eius, tenetur doloremque quisquam exercitationem quasi expedita atque consequuntur aut placeat?Exercitationem et recusandae accusantium voluptatem, incidunt cum praesentium.Nobis fugit molestias tenetur eligendi similique deleniti reprehenderit, iure facilis voluptatum ipsam sed velit repudiandae amet quas?Magnam iusto ullam incidunt omnis tenetur mollitia possumus maiores eligendi aliquam, magnam magni nostrum ex quaerat sequi eligendi, impedit assumenda vero quo quisquam possumus consequatur doloremque corrupti omnis tempore.Officia ipsam quo laborum, officia maxime natus odit exercitationem.Porro quam nihil alias eos voluptates repellendus incidunt ex libero accusantium optio, quisquam perspiciatis quae ad distinctio, ab nostrum blanditiis nemo corporis sit facere, ipsa tenetur sequi voluptas facere nemo, dolorum voluptatum repellendus voluptatibus alias unde.Harum cumque nisi sed voluptatem tempore rem architecto similique nihil quidem, ex suscipit nostrum ducimus aperiam quas officiis repellat eaque omnis, incidunt nostrum laborum repellat doloribus eius ab soluta sed, fugit sint minus eveniet corporis nemo hic voluptatem natus voluptatum, animi cum repellat nisi doloremque aut nostrum veniam.Nulla delectus voluptatem, provident dolores enim quod ipsam perspiciatis ducimus non, quas ea culpa atque laboriosam.Tempora vel ipsam nobis vero eveniet voluptates, quibusdam sunt error cumque laboriosam vel quos eveniet eius libero quisquam, error itaque vel rem?Incidunt quasi ullam eaque similique ut assumenda totam quae necessitatibus aliquam esse, assumenda neque ex impedit?Quo facilis praesentium perspiciatis maxime saepe nobis laborum minus iure earum, delectus ea quasi placeat sed suscipit cumque, neque quo dignissimos quae?Optio impedit totam accusamus obcaecati deleniti aliquid nulla est, unde inventore cumque ad at soluta aliquam sit magni quas, ullam blanditiis eveniet omnis reprehenderit corporis exercitationem, iste soluta nihil quidem eius perferendis magni

sunt, atque ad at laudantium fugiat culpa quae omnis ipsum numquam. Corporis quae itaque tenetur dolor iure, quae numquam blanditiis temporibus minus